

DBMS Lab: SQL Joins and Transactions

Part A: Create Tables

- CREATE TABLE roles(role_id INT PRIMARY KEY, role_name VARCHAR(50));
- CREATE TABLE users(user_id INT PRIMARY KEY,name VARCHAR(50),email VARCHAR(50),role_id INT);
- CREATE TABLE departments(dept_id INT PRIMARY KEY,dept_name VARCHAR(50));
- CREATE TABLE employees(emp_id INT PRIMARY KEY,emp_name VARCHAR(50),dept_id INT,manager_id INT,salary DECIMAL(10,2));
- CREATE TABLE customers(customer_id INT PRIMARY KEY,customer_name VARCHAR(50),city VARCHAR(50));
- CREATE TABLE orders(order_id INT PRIMARY KEY,customer_id INT,order_date DATE,total_amount DECIMAL(10,2));
- CREATE TABLE accounts(account_no INT PRIMARY KEY,holder_name VARCHAR(50),balance DECIMAL(10,2));

OUTPUT:

```
Microsoft Windows [Version 10.0.26200.9445]
(c) Microsoft Corporation. All rights reserved.

C:\Users\rohit>cd C:\Program Files\MySQL\MySQL Server 8.0\bin

C:\Program Files\MySQL\MySQL Server 8.0\bin>mysql -u root -p
Enter password: ****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 11
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> show databases;
+-----+
| Database |
+-----+
| bookflow_db |
| company_db |
| companydb1 |
| db2 |
| employees_1 |
| information_schema |
| library_db |
| local_service_booking_db |
| mysql |
| performance_schema |
| studentdb |
| sys |
| transactions1 |
+-----+
13 rows in set (0.00 sec)

mysql> DROP DATABASE transactions1;
Query OK, 0 rows affected (0.05 sec)
```

```
mysql> CREATE DATABASE Transactions1;
Query OK, 1 row affected (0.02 sec)

mysql> USE Transactions1;
Database changed
mysql> CREATE TABLE roles(role_id INT PRIMARY KEY, role_name VARCHAR(50));
Query OK, 0 rows affected (0.06 sec)

mysql> CREATE TABLE users(user_id INT PRIMARY KEY, name VARCHAR(50), email VARCHAR(50), role_id INT);
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE departments(dept_id INT PRIMARY KEY, dept_name VARCHAR(50));
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE employees(emp_id INT PRIMARY KEY, emp_name VARCHAR(50), dept_id INT, manager_id INT, salary DECIMAL(10,2));
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE customers(customer_id INT PRIMARY KEY, customer_name VARCHAR(50), city VARCHAR(50));
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE orders(order_id INT PRIMARY KEY, customer_id INT, order_date DATE, total_amount DECIMAL(10,2));
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE accounts(account_no INT PRIMARY KEY, holder_name VARCHAR(50), balance DECIMAL(10,2));
Query OK, 0 rows affected (0.04 sec)
```

Q1. Display employee names with their department names using INNER JOIN.

```
SELECT emp_name,dept_name FROM employees e INNER JOIN departments d ON  
e.dept_id=d.dept_id;
```

Q2. Display all users with their assigned roles using LEFT JOIN.

```
SELECT name,email,role_name FROM users u LEFT JOIN roles r ON  
u.role_id=r.role_id;
```

Q3. Display all roles even if no users are assigned using RIGHT JOIN.

```
SELECT name,role_name FROM users u RIGHT JOIN roles r ON u.role_id=r.role_id;
```

Q4. Simulate FULL OUTER JOIN in MySQL.

```
SELECT name,email,role_name FROM users u LEFT JOIN roles r ON  
u.role_id=r.role_id UNION SELECT name,email,role_name FROM users u RIGHT  
JOIN roles r ON u.role_id=r.role_id;
```

Q5. Display employee and manager names using SELF JOIN.

```
SELECT e.emp_name Employee,m.emp_name Manager FROM employees e LEFT  
JOIN employees m ON e.manager_id=m.emp_id;
```

Q6. Display Order ID, Customer Name, City and Total Amount.

```
SELECT o.order_id,customer_name,city,total_amount FROM customers c JOIN orders o  
ON c.customer_id=o.customer_id;
```

Q7. Find department-wise employee count, average salary and maximum salary.

```
SELECT dept_name,COUNT(*),AVG(salary),MAX(salary) FROM employees e JOIN  
departments d ON e.dept_id=d.dept_id GROUP BY dept_name;
```

Q8. Display employees earning >50000 in CSE.

```
SELECT emp_name,salary,dept_name FROM employees e JOIN departments d ON  
e.dept_id=d.dept_id WHERE salary>50000 AND dept_name='CSE';
```

Q9. Display customer-wise total purchase in descending order.

```
SELECT customer_name,SUM(total_amount) TotalPurchase FROM customers c JOIN  
orders o ON c.customer_id=o.customer_id GROUP BY customer_name ORDER BY  
TotalPurchase DESC;
```

Q10. Create Students, Courses, Faculty, Enrollments and display Student, Course, Faculty using joins.

```
SELECT student_name,course_name,faculty_name FROM students s JOIN enrollments  
e ON s.student_id=e.student_id JOIN courses c ON e.course_id=c.course_id JOIN  
faculty f ON c.faculty_id=f.faculty_id;
```

OUTPUT:

```
mysql> SELECT emp_name,dept_name FROM employees e INNER JOIN departments d ON e.dept_id=d.dept_id;
Empty set (0.02 sec)

mysql> SELECT name,email,role_name FROM users u LEFT JOIN roles r ON u.role_id=r.role_id;
Empty set (0.01 sec)

mysql> SELECT name,role_name FROM users u RIGHT JOIN roles r ON u.role_id=r.role_id;
Empty set (0.00 sec)

mysql> SELECT name,email,role_name FROM users u LEFT JOIN roles r ON u.role_id=r.role_id UNION SELECT name,email,role_name FROM users u RIGHT JOIN roles r ON u.role_id=r.role_id;
Empty set (0.02 sec)

mysql> SELECT e.emp_name Employee,m.emp_name Manager FROM employees e LEFT JOIN employees m ON e.manager_id=m.emp_id;
Empty set (0.00 sec)

mysql> SELECT o.order_id,customer_name,city,total_amount FROM customers c JOIN orders o ON c.customer_id=o.customer_id;
Empty set (0.00 sec)

mysql> SELECT dept_name,COUNT(*),AVG(salary),MAX(salary) FROM employees e JOIN departments d ON e.dept_id=d.dept_id GROUP BY dept_name;
Empty set (0.02 sec)

mysql> SELECT emp_name,salary,dept_name FROM employees e JOIN departments d ON e.dept_id=d.dept_id WHERE salary>50000 AND dept_name='CSE';
Empty set (0.01 sec)

mysql> SELECT customer_name,SUM(total_amount) TotalPurchase FROM customers c JOIN orders o ON c.customer_id=o.customer_id GROUP BY customer_name ORDER BY TotalPurchase DESC;
Empty set (0.00 sec)

mysql> SELECT student_name,course_name,faculty_name FROM students s JOIN enrollments e ON s.student_id=e.student_id JOIN courses c ON e.course_id=c.course_id JOIN faculty f ON c.faculty_id=f.faculty_id;
ERROR 1146 (42S02): Table 'transactions1.students' doesn't exist
mysql> START TRANSACTION; INSERT INTO accounts VALUES(104,'Anil',25000); COMMIT;
Query OK, 0 rows affected (0.00 sec)

Query OK, 1 row affected (0.02 sec)

Query OK, 0 rows affected (0.01 sec)
```

Part B: Transactions

Q11. Insert a new account and COMMIT.

START TRANSACTION; INSERT INTO accounts VALUES(104,'Anil',25000);
COMMIT;

Q12. Insert a record and ROLLBACK.

START TRANSACTION; INSERT INTO accounts VALUES(105,'Ravi',10000);
ROLLBACK;

Q13. Transfer ₹5000 from Account 101 to 102.

START TRANSACTION; UPDATE accounts SET balance=balance-5000 WHERE
account_no=101; UPDATE accounts SET balance=balance+5000 WHERE
account_no=102; COMMIT;

Q14. Simulate failed transaction and rollback.

START TRANSACTION; UPDATE accounts SET balance=balance-3000 WHERE
account_no=101; UPDATE accounts SET balance=balance+3000 WHERE
account_no=999; ROLLBACK;

Q15. Demonstrate SAVEPOINT.

START TRANSACTION; UPDATE accounts SET balance=balance+2000 WHERE account_no=101; SAVEPOINT s1; UPDATE accounts SET balance=balance-1000 WHERE account_no=101; ROLLBACK TO s1; COMMIT;

OUTPUT:

```
mysql> START TRANSACTION; INSERT INTO accounts VALUES(105,'Ravi',10000); ROLLBACK;
Query OK, 0 rows affected (0.00 sec)

Query OK, 1 row affected (0.00 sec)

Query OK, 0 rows affected (0.02 sec)

mysql> START TRANSACTION; UPDATE accounts SET balance=balance-5000 WHERE account_no=101; UPDATE accounts SET balance=balance+5000 WHERE account_no=102; COMMIT;
Query OK, 0 rows affected (0.00 sec)

Query OK, 0 rows affected (0.01 sec)
Rows matched: 0 Changed: 0 Warnings: 0

Query OK, 0 rows affected (0.00 sec)
Rows matched: 0 Changed: 0 Warnings: 0

Query OK, 0 rows affected (0.00 sec)

mysql> START TRANSACTION; UPDATE accounts SET balance=balance-3000 WHERE account_no=101; UPDATE accounts SET balance=balance+3000 WHERE account_no=999; ROLLBACK;
Query OK, 0 rows affected (0.00 sec)

Query OK, 0 rows affected (0.00 sec)
Rows matched: 0 Changed: 0 Warnings: 0

Query OK, 0 rows affected (0.00 sec)
Rows matched: 0 Changed: 0 Warnings: 0

Query OK, 0 rows affected (0.00 sec)

mysql> START TRANSACTION; UPDATE accounts SET balance=balance+2000 WHERE account_no=101; SAVEPOINT s1; UPDATE accounts SET balance=balance-1000 WHERE account_no=101; ROLLBACK TO s1; COMMIT;
Query OK, 0 rows affected (0.00 sec)

Query OK, 0 rows affected (0.00 sec)
Rows matched: 0 Changed: 0 Warnings: 0

Query OK, 0 rows affected (0.00 sec)
```